Computability: Recitation 9

June 1^{th}, 2026

1 Verifiers

Reminder, NP definition:

$$\text{NP} = \bigcup_{k=0}^{\infty} \text{NTIME}(n^k)$$

Definition 1 (Polynomial Verifier)**.** *A verifier for a language L is a deterministic TM V which given (w, c), with c polynomial in $|w|$, runs in polynomial time in $|w|$ and*

$$\forall w \in L, \exists c \quad s.t. \quad V(w, c) = q_{acc}$$

$$\forall w \notin L, \forall c \;\; V(w, c) = q_{rej}$$

c is called a witness.

Proposition 2 (Equivalent definition of NP)**.** *A language L can be recognized by an NTM in polynomial time iff there is a polynomial verifier for L.*

Proof. Proved in the lecture. □

2 U-ST-HAMPATH

We define the language

$$\text{U-ST-HAMPATH} = \{\langle G, s, t\rangle : G \text{ is undirected, and there exists a Hamilton path from } s \text{ to } t\}$$

Proposition 3. U-ST-HAMPATH *is NP-complete.*

Proof. First we prove that U-ST-HAMPATH $\in$ NP.

We show that there exists a polynomial-time verifier.
A verifier V gets as input $w = \langle G, s, t\rangle$, a witness $c = \langle v_1, ..., v_n\rangle$ - a sequence of $n = |V|$ vertices, and performs the following:

1. Checks if the first vertex is s, and if the last vertex is t
2. Checks that all nodes are different (no repetitions)
3. Validates that there's an edge between every 2 adjacent vertices.

The witness c is polynomial in the input size $|\langle G, s, t\rangle|$, and each operation can be performed in polynomial time.

We now prove that U-ST-HAMPATH is NP-hard.
In the previous recitation we proved 3-SAT$\leq_p$ D-ST-HAMPATH. As 3-SAT is NP-hard then so is D-ST-HAMPATH.
We now show that U-ST-HAMPATH is NP-hard by a reduction D-ST-HAMPATH $\leq_p$ U-ST-HAMPATH.

Construction: Let $\langle G, s, t\rangle$ be an input for D-ST-HAMPATH, with $G = \langle V, E\rangle$. The reduction T constructs the input $\langle G', s_{in}, t_{out}\rangle$ to U-ST-HAMPATH, where G' is defined as follows.

1. For every vertex $v \in V$ T introduces the vertices v_{in}, v_{mid} and v_{out}, and the edges $\{v_{in}, v_{mid}\}$ and $\{v_{mid}, v_{out}\}$.
2. For every edge $(u, v) \in E$ we define the edge $\{u_{out}, v_{in}\}$.

Thus, $G' = \langle V', E'\rangle$ where $V' = \{v_{in}, v_{out}, v_{mid} : v \in V\}$ and
$E' = \{\{v_{in}, v_{mid}\}, \{v_{mid}, v_{out}\} : v \in V\} \cup \{\{u_{out}, v_{in}\} : (u, v) \in E\}$.

Runtime: Clearly the reduction is polynomial, since the size of G' is 3 times the size of G, and computing it is straightforward.

Correctness: For the easy direction, assume $\langle G, s, t\rangle \in$ D-ST-HAMPATH. Then, let $s, u^1, u^2, ..., u^k, t$ be a directed Hamilton path in G. The path induces the following path in G':

$$s_{in}, s_{mid}, s_{out}, u^1_{in}, u^1_{mid}, u^1_{out}, ..., u^k_{in}, u^k_{mid}, u^k_{out}, t_{in}, t_{mid}, t_{out}$$

Since the original path contained all the vertices, so does the induced path. Thus, $\langle G', s_{in}, t_{out}\rangle \in$ U-ST-HAMPATH.

For the hard direction, assume that $\langle G', s_{in}, t_{out}\rangle \in$ U-ST-HAMPATH. Thus, there is a Hamiltonian path in G', but we do not know what this path "looks like". We proceed with the following claim.

Claim 4. *A Hamiltonian path that starts at s_{in} and ends at t_{out} does not contain a directed traversal of the form (v_{in}, u_{out}). That is, we never go "backward" on edges.*

The proof of this claim is by contradiction. Assume that there is such a directed traversal, and let (v_{in}, u_{out}) be the first one in the path. Since the path is Hamiltonian, we must visit v_{mid} at some point. If we already visited it, then we must have visited it from v_{out} (since we are just now visiting v_{in}). But how did we reach v_{out}? it must have been from some x_{in} (since these are the only possible edges left). This is a contradiction to (v_{in}, u_{out}) being the first such pair.

Thus, we must visit v_{mid} after u_{out}, but that means we reach it from v_{out}, at which point we get stuck, and in particular we cannot end in t_{out}.

We conclude that the edges in the path are of the forms $(v_{in}, v_{mid}), (v_{mid}, v_{out})$ and (v_{out}, u_{in}). Thus, the Hamiltonian path is of the form

$$s_{in}, s_{mid}, s_{out}, u^1_{in}, u^1_{mid}, u^1_{out}, ..., u^k_{in}, u^k_{mid}, u^k_{out}, t_{in}, t_{mid}, t_{out}$$

which can be easily mapped to a Hamiltonian path in G.

□

3 3-COLORING

Given a graph $G = (V, E)$, the k-COLORING problem is to validate whether or not it is possible to color all vertices in k different colors such that no 2 neighbouring vertices have the same color.

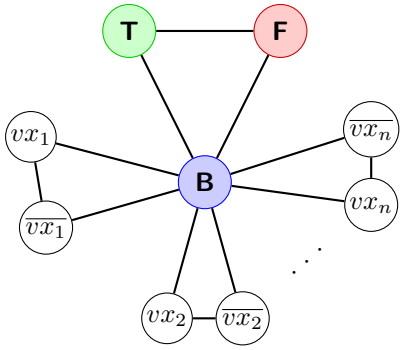
$$\text{3-COLORING} = \{\langle G\rangle : \text{There's a valid 3-coloring of } G\}$$

Proposition 5. *3-COLORING is NP-complete.*

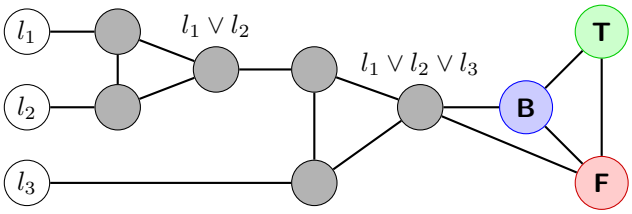
Proof. We first show that 3-COLORING$\in$ NP by showing a verifier V.
Given a graph $\langle G\rangle$, and a coloring c, V checks that c is a valid coloring by going over all edges in E, and verifies that the vertices of the edge have different colors. This can be done in polynomial time.
Now we prove that 3-SAT $\leq_p$ 3-COLORING, thus proving hardness in NP.
Let $\langle\varphi\rangle$ be a 3-CNF formula with $x_1, ..., x_n$ variables, and $c_1, ..., c_m$ clauses. The reduction creates the following graph:

1. Create 3 vertices - T, F, B, and connect them all together.

2. For each variable $x_i \in \varphi$, create 2 vertices $vx_i, \overline{vx_i}$, add an edge between them - $\{vx_i, \overline{vx_i}\}$, and an edge from each of them to $B : \{vx_i, B\}\,\{\overline{vx_i}, B\}$.



3. For each clause $c_j = (l_1 \vee l_2 \vee l_3)$ add triple or gadget graph, connect to relevant literals vertices, and connect the output to B and to F (forcing it to be T)



Notice, if l_1, l_2, l_3 are all false, then the triple or gadget output must be colored false, but it is also connected to F, so it is an invalid coloring. On the other hand, if at least one of them is colored true then there exists a valid coloring.

Correctness: $\varphi \in$ 3-SAT $\implies$ φ is satisfiable. Let $A(x_i) \to \{T, F\}$ be a satisfying assignment. If x_i is assigned T, we color vx_i green $= T$, and $\overline{vx_i}$ red $= F$. Otherwise we assign the opposite. This is valid as they are connected to B, and to each other, 3 different colors so far. Since φ is satisfiable, each clause c_j is satisfiable, so at least one of the literals is true, and from the or gadget observation, it is 3-colorable.
$\langle G \rangle \in$ 3-COLORING. We construct a satisfying assignment through the color of the vertices vx_i (we call the color of T green, the color of F red and the color of B blue). If it is colored green we assign true, otherwise false. If by contradiction this is not a satisfying assignment, then there is a clause c_j which is not satisfied. As such, l_1^j, l_2^j, l_3^j are all false, and colored false in the graph. In this case this means the clause triple or gadget is not 3 colorable, thus the entire graph is not 3 colorable in contradiction. So the assignment must be a satisfying assignment.
Complexity: For each variable we construct 2 nodes, and connect 3 edges, this is polynomial in n. For each clause we create an or gadget which is a fixed size of nodes and edges. There are m gadgets, so this too is polynomial in the input size. □

4 2-SAT

In the previous recitation we have seen that k-SAT $\leq_p$ 3-SAT for $k > 3$. It is trivial to show that 2-SAT $\leq_p$ 3-SAT, simply by duplicating a literal to each clause.
However, this does not mean that 2-SAT is a hard problem.

Proposition 6. *2-SAT* $\in P$

Proof. It's important to notice that if $(x_i \vee x_j)$ is a clause, and $x_i = F$ then it must be that $x_j = T$.
Construction: We define a TM M that decides 2-SAT. Given $\langle \varphi \rangle \in$2-CNF with n variables $x_1, ..., x_n$, and m clauses $c_1, ..., c_m$, M operates as follows:

1. for each unassigned variable x_i
   (a) attempt $x_i = True$
   (b) while new assignments are made
      i. for each clause $(l_1 \vee l_2)$

A. if one literal is $False$ under current assignment and the other is unassigned, assign it to make it $True$.

B. else, if it is already assigned with value $False$ - contradiction.

(c) if contradiction was found:

i. undo current loop assignments

ii. Repeat the while loop with $x_i = False$

iii. if contradiction found again - Reject

2. all variables assigned with no contradiction - Accept

Example - satisfiable

$$(x_1 \vee x_2) \wedge (\overline{x_3} \vee x_4) \wedge (\overline{x_1} \vee x_3) \wedge (\overline{x_4} \vee \overline{x_1})$$

$$x_1 = T,\ \underbrace{(x_1 \vee x_2)}_{T} \wedge (\overline{x_3} \vee x_4) \wedge \underbrace{(\overline{x_1} \vee x_3)}_{x_3=T} \wedge \underbrace{(\overline{x_4} \vee \overline{x_1})}_{x_4=F}$$

$$x_1 = T,\ \underbrace{(x_1 \vee x_2)}_{T} \wedge \underbrace{(\overline{x_3} \vee x_4)}_{x_4=T\text{: contradiction}} \wedge \underbrace{(\overline{x_1} \vee x_3)}_{x_3=T} \wedge \underbrace{(\overline{x_4} \vee \overline{x_1})}_{x_4=F}$$

$$x_1 = F,\ \underbrace{(x_1 \vee x_2)}_{x_2=T} \wedge (\overline{x_3} \vee x_4) \wedge \underbrace{(\overline{x_1} \vee x_3)}_{T} \wedge \underbrace{(\overline{x_4} \vee \overline{x_1})}_{T}$$

Subset: $(\overline{x_3} \vee x_4)$

$$x_3 = T,\ \underbrace{(\overline{x_3} \vee x_4)}_{x_4=T}$$

Example - unsatisfiable

$$(x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (\overline{x_2} \vee x_3) \wedge (\overline{x_3} \vee x_2) \wedge (\overline{x_2} \vee \overline{x_2})$$

$$x_1 = T,\ \underbrace{(x_1 \vee x_2)}_{T} \wedge (x_2 \vee x_3) \wedge (\overline{x_2} \vee x_3) \wedge (\overline{x_3} \vee x_2) \wedge (\overline{x_2} \vee \overline{x_2})$$

Subset: $(x_2 \vee x_3) \wedge (\overline{x_2} \vee x_3) \wedge (\overline{x_3} \vee x_2) \wedge (\overline{x_2} \vee \overline{x_2})$

$$x_2 = T,\ (x_2 \vee x_3) \wedge (\overline{x_2} \vee x_3) \wedge (\overline{x_3} \vee x_2) \wedge \underbrace{(\overline{x_2} \vee \overline{x_2})}_{x_2=F\text{: contradiction}}$$

$$x_2 = F,\ \underbrace{(x_2 \vee x_3)}_{x_3=T} \wedge \underbrace{(\overline{x_2} \vee x_3)}_{T} \wedge \underbrace{(\overline{x_3} \vee x_2)}_{x_3=F\text{: contradiction}} \wedge \underbrace{(\overline{x_2} \vee \overline{x_2})}_{T}$$

Correctness: If M accepts, that means all variables were assigned, and no contradiction was made. Thus, all clauses are satisfied $\implies$ this is a satisfying assignment to $\varphi \implies \langle\varphi\rangle \in L(M)$.
In the other direction, first M rejects only if there is a contradiction to both $x_i = True$ and $x_i = False$ at some iteration in the outer loop (step 1). Notice that if we reach a new loop iteration in step 1, then some variables were assigned. All clauses affected by the assigned variables are satisfied (otherwise we would keep assigning or reach contradiction). Thus, we are left with a smaller problem containing the clauses which aren't affected at all by the assigned variables, and all variables unassigned yet. So, if an assignment to a variable x_i is set as true, and there is a contradiction in the sub-problem, it is definitely also a contradiction in the larger problem. Then if both $x_i = True$ and $x_i = False$ are contradictions, there necessarily isn't a satisfying assignment to the original formula $\implies \langle\varphi\rangle \notin$ 2-SAT.
Complexity: The outer loop (1.) runs for at most n iterations. At each iteration of the inner loop ((b)), at least 1 more assignment is made, so it is run at most n times. The most inner loop (i.) runs over m clauses, and the evaluations inside can be done in polynomial time (we might have to loop over all variables again for that). Step (c) re-performs the outer loop with $x_i = False$, so it is still polynomial. All operations can be performed in polynomial time, and are run a polynomial number of times, thus M runs in polynomial time in total. □